1. Design Patterns

1.1 Factory Pattern

Factory Method

[The Factory Method Pattern Explained and Implemented in Java | Creational Design Patterns | Geekific]

1.1.1 Gebruik

Gebruik dit als je **één object** wilt maken, maar de keuze voor de precieze versie pas door een subklasse wilt laten maken.

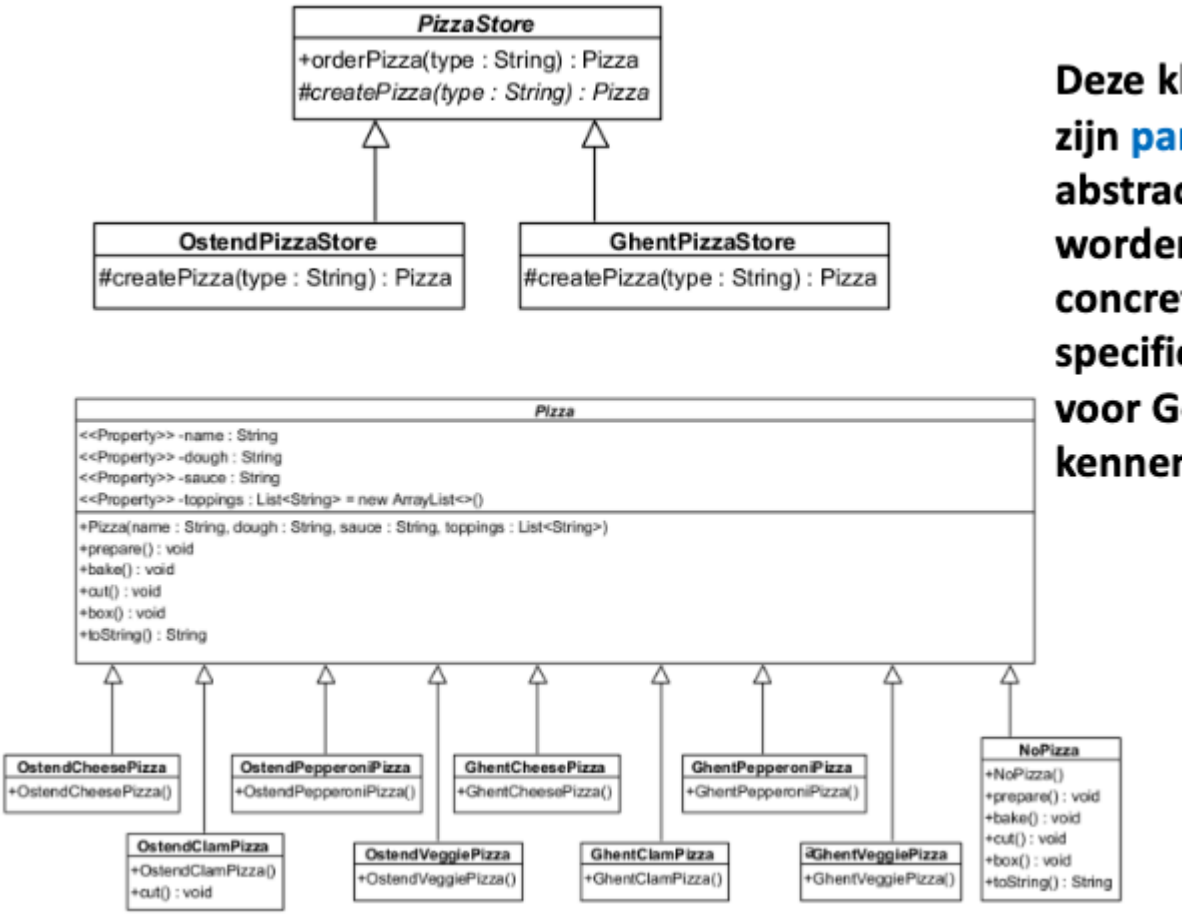
1.1.2 Voorbeeld

Pizza-keten die bepaalde pizzas enkel op bepaalde vestigingen maakt (Gent (standaard), Oostende (vis), ...)

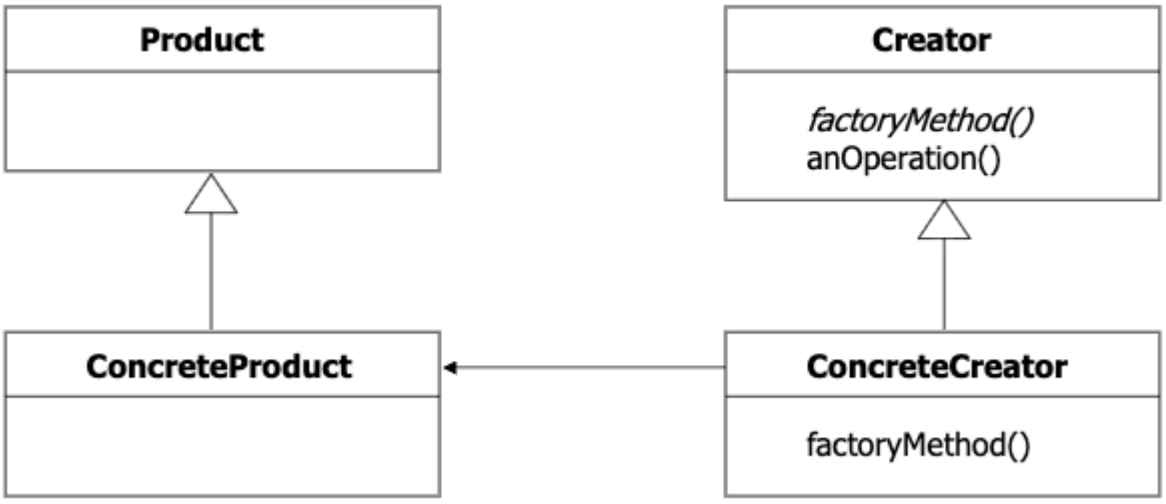
1.1.3 Stappen

- **Stap 1:** Maak een **interface of abstracte klasse** voor het **Product**
- **Stap 2:** Maak de **Concrete Producten** die deze interface implementeren
- **Stap 3:** Maak een **abstracte Creator klasse**
- **Stap 4:** Declareer in deze Creator de **abstracte Factory Method**
- **Stap 5:** Schrijf in de Creator een gewone methode die de Factory Method aanroept om een object te krijgen en er acties op uitvoert.
- **Stap 6:** Maak **Concrete Creators** (subklassen) die de Factory Method overschrijven om het specifieke object terug te geven.

1.1.4 UML



ALGEMENE UML



1.1.5 Code

Maak de oefening(en) op chamilo

Abstract Factory

VIDEO
CHAMILO UITLEG

1.2.1 Gebruik

Gebruik dit als je een hele **set van bijpassende producten** (een familie, zoals alle ingrediënten voor een pizza) tegelijk moet maken zonder de exacte klassen te hoeven kennen.

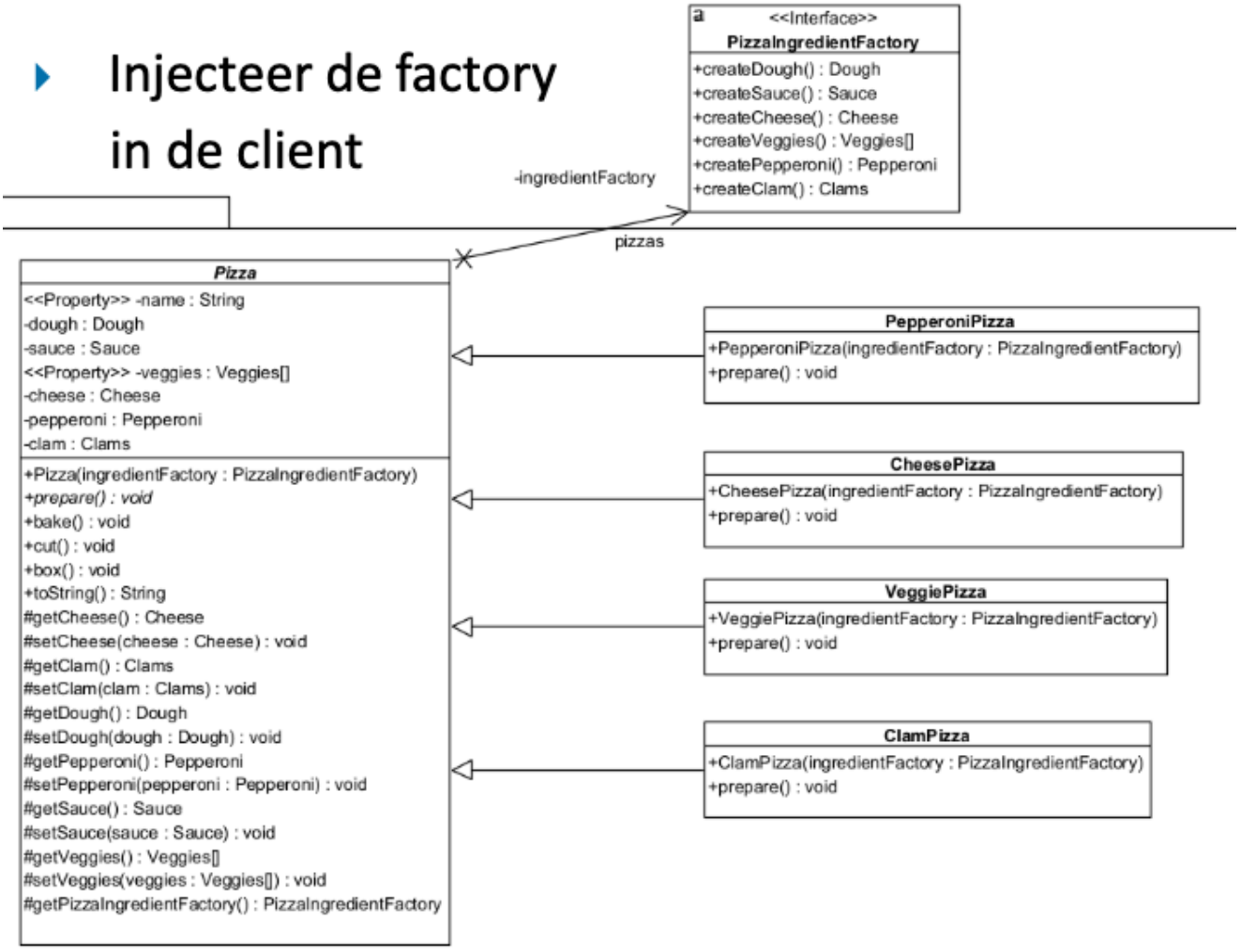
1.2.2 Voorbeeld

Interface-fabriek die knoppen en vensters maakt voor verschillende besturingssystemen (Windows en Mac):
Je code maakt een knop en de fabriek zorgt ervoor dat deze wordt aangepast aan het besturingssysteem.

1.2.3 Stappen

- **Stap 1:** Maak per gedrag een interface
- **Stap 2:** Maak voor elke familie de concrete klassen aan
- **Stap 3:** Maak een abstracte factory. Voeg een create method toe per soort object (kolom).
- **Stap 4:** Maak per familie een concrete factory
- **Stap 5:** Injecteer de factoryin de client

1.2.4 UML



1.2.5 Code

Maak de oefening(en) op chamilo

1.2 Builder Pattern

[Builder Design Pattern: Easy Guide for Beginners]
[The Builder Pattern Explained and Implemented in Java | Creational Design Patterns | Geekific]
[Builder Design Pattern in Java Theory]

1.2.1 Gebruik

Gebruik dit als je een **complex object stap voor stap** wilt opbouwen.
Ideaal als je object heel veel verschillende configuraties kan hebben en geen messy constructor met 10 parameters wilt.

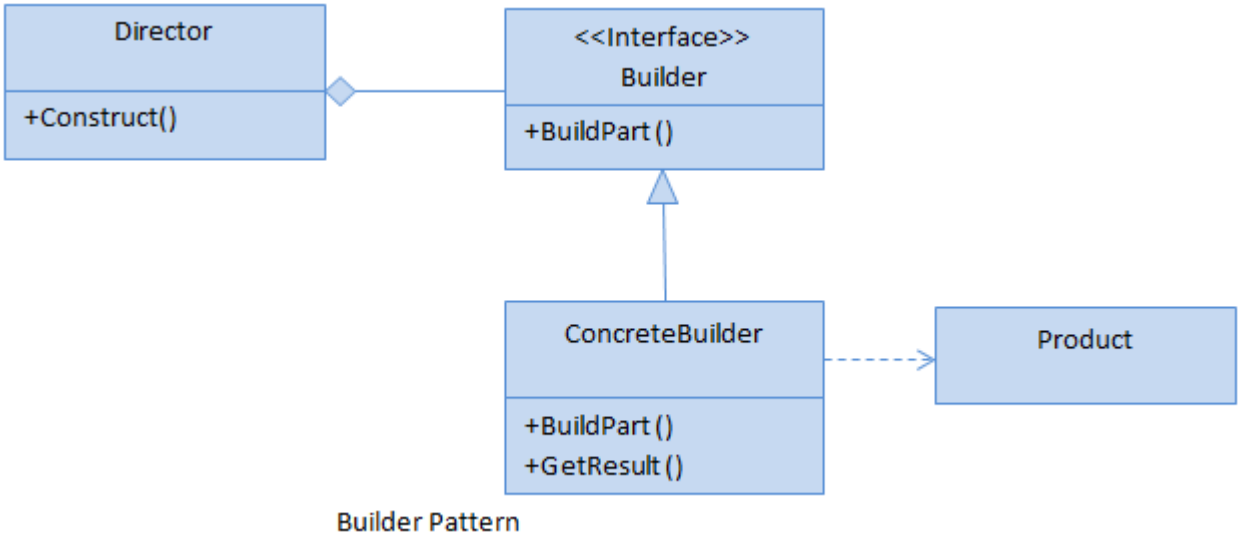
1.2.2 Voorbeeld

Een **PC samenstellen**: Je kunt kiezen welke CPU, hoeveel RAM, welke videokaart en of je wel of geen RGB-verlichting wilt. In plaats van één gigantische constructor, gebruik je een builder waarbij je zegt: `.addCPU("Ryzen 9")`, `.addRAM(32)`, `.enableRGB()`, en pas aan het einde `.build()`.

1.2.3 Stappen

- **Stap 1:** Maak de Builder interface
- **Stap 2:** Maak de implementatieklassen (Concrete Builders)
- **Stap 3:** Maak de Product klasse
- **Stap 4:** Maak de Director klasse

1.2.4 UML



Builder Pattern

1.2.5 Code

Maak de oefening(en) op chamilo

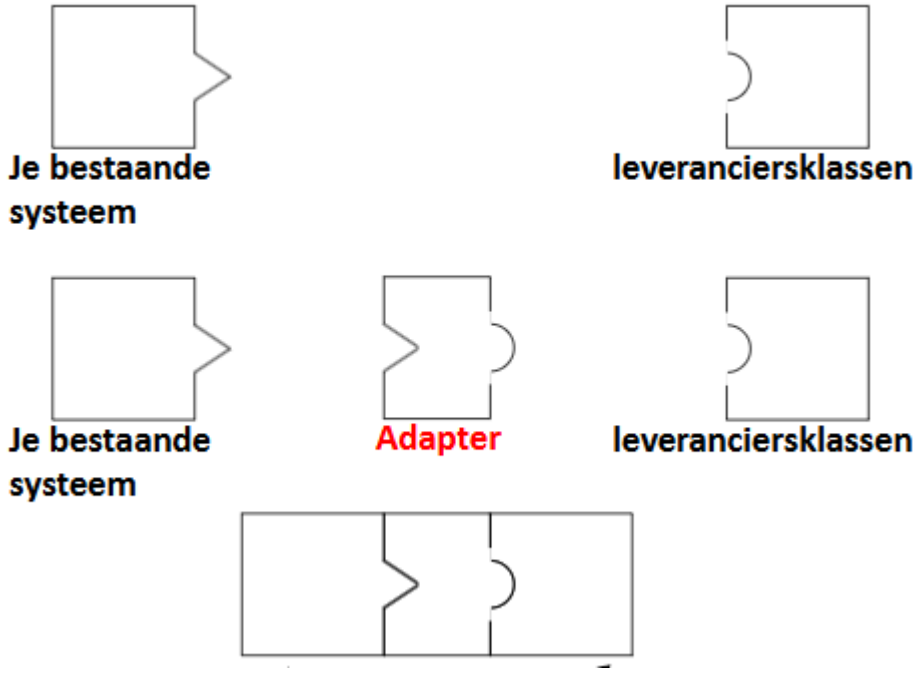
1.3 Adapter Pattern

[The Adapter Pattern Explained and Implemented in Java]
[Adapter Design Pattern in Java Theory]
[Adapter Design Pattern in Java]

1.3.1 Gebruik

Wordt gebruikt wanneer je iets moet omzetten in iets anders

1.3.2 Voorbeeld



Adapter pattern is zoals een een male USB A naar female USB C adapter. Het is nodig om de twee apparaten te verbinden want het ene apparaat heeft enkel USB A kabel en het andere apparaat heeft enkel een USB C poort.

Ander voorbeeld is een tolk. De ene persoon praat Spaans en de andere verstaat geen Spaans. Ze huren een tolk die Spaans vertaalt naar Nederlands. De Spanjaard praat tegen de tolk en de tolk zet dit om naar het Nederlands.

Ander voorbeeld is hieronder

Voor een Kalkoen toe te voegen aan het project dat oorspronkelijk enkel voor een Eend was gemaakt, heb je de KalkoenAdapter nodig die een Eend omzet in een Kalkoen




```java
@AllArgsConstructor
public class TurkeyAdapter implements Duck {

    private Turkey turkey;

    public void quack() {
        turkey.gobble();
    }

    public void fly() {
        for (int i = 0; i < 5; i++) {
            turkey.fly();
        }
    }

}
public class AdapterStartUp {

    public static void main(String[] args) {
        new AdapterStartUp();
    }

    public AdapterStartUp() {
        MallardDuck duck = new MallardDuck();
        testDuck(duck);

        WildTurkey turkey = new WildTurkey();
        Duck turkeyAdapter = new TurkeyAdapter(turkey);
        testDuck(turkeyAdapter);
    }

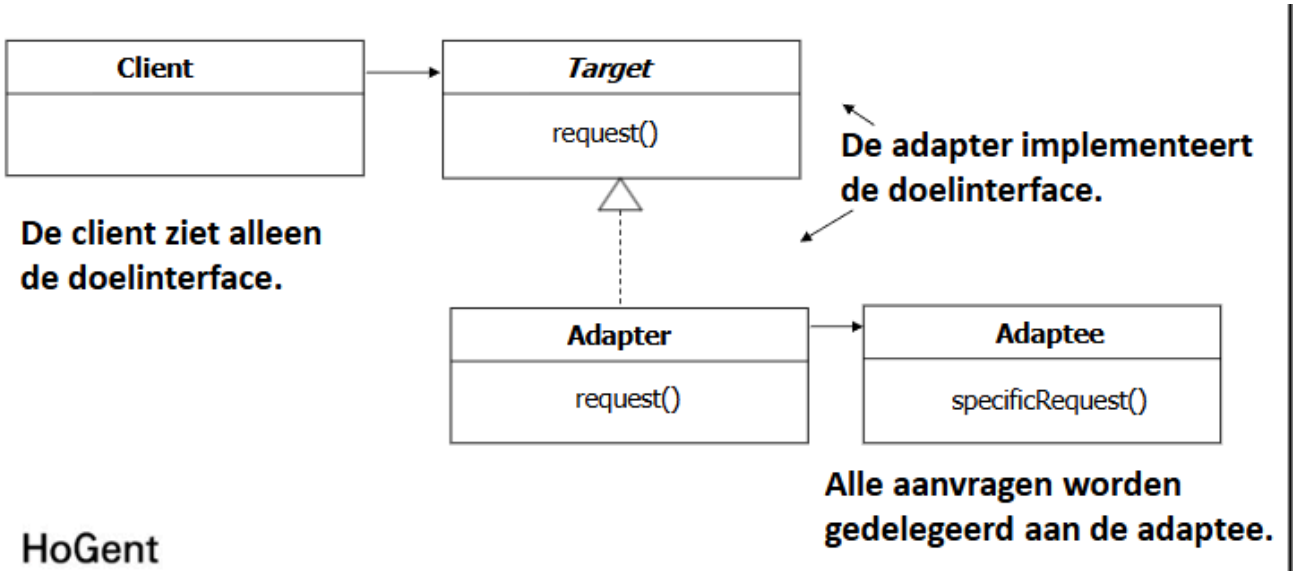
    private void testDuck(Duck duck) {
        duck.quack();
        duck.fly();
    }
}
```

```
Quack
I'm flying
Gobble gobble
I'm flying a short distance
I'm flying a short distance
I'm flying a short distance
I'm flying a short distance
I'm flying a short distance
```

1.3.3 Stappen

- **Stap 1:** Maak de Target interface die de gewenste functionaliteit definieert voor de client.
- **Stap 2:** Identificeer de Adaptee klasse die de bestaande, maar incompatibele code bevat.
- **Stap 3:** Maak de Adapter klasse die de Target interface implementeert.
- **Stap 4:** Voeg een referentie van de Adaptee toe aan de Adapter klasse middels compositie.
- **Stap 5:** Koppel de Target methoden in de Adapter aan de specifieke logica van de Adaptee.

1.3.4 Algemene UML



1.3.5 Code

Maak de oefening(en) op chamilo

1.4 Singleton Pattern

[The Singleton Pattern Explained and Implemented in Java | Creational Design Patterns | GeekificSingleton Design Pattern: Easy Guide for Beginners]

1.4.1 Gebruik

- Er bestaat maar 1 instantie van
- Het object is overal beschikbaar in de applicatie
- Ipv. elke keer het object opnieuw te maken, wordt het object hergebruikt
- Wordt gebruikt voor multithreading (niet zeker of we dit moeten kennen of niet)

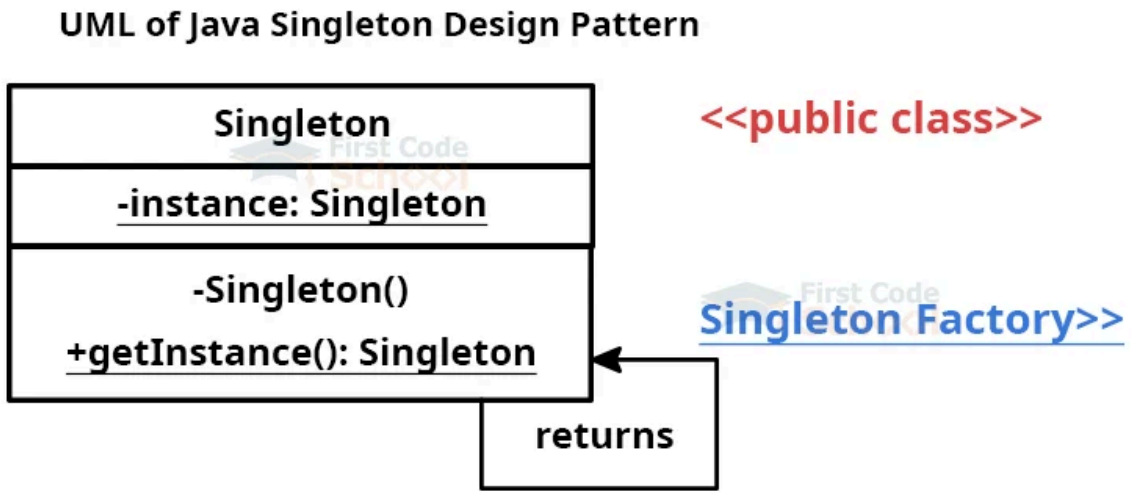
1.4.2 Voorbeeld

Een land kan maar 1 president hebben. Meer dan 1 president kan niet. Het wordt dus gelimiteerd op 1!

1.4.3 Stappen

- **Stap 1:** Maak de klasse aan die slechts één instantie mag hebben (de Singleton klasse).
- **Stap 2:** Maak een **private static** variabele van hetzelfde klassetype aan binnen de klasse om de enige instantie op te slaan.
- **Stap 3:** Zet de **constructor op private**, zodat de klasse niet van buitenaf geïnstantieerd kan worden met het `new` keyword.
- **Stap 4:** Schrijf een **public static** methode (meestal `getInstance()`) die fungeert als het centrale toegangspunt tot de instantie.
- **Stap 5:** Implementeer logica in deze methode die controleert of de instantie al bestaat: zo ja, geef deze terug; zo nee, maak de instantie aan en geef deze terug (**lazy initialization**).

1.4.4 UML



1.4.5 Code

Maak de oefening(en) op chamilo

1.5 Iterator Pattern

The Iterator Pattern Explained and Implemented in Java | Behavioral Design Patterns | Geekific
Iterator Design Pattern: Easy Guide for Beginners
CHAMILO

1.5.1 Gebruik

- Biedt een uniforme manier om over verschillende soorten collecties te itereren.
- Ontkoppelt de client (de "Waitress") van de concrete implementatie van de data (zoals `ArrayList` of `Array`).
- Maakt het mogelijk om polymorfisme te gebruiken bij het doorlopen van verzamelingen.

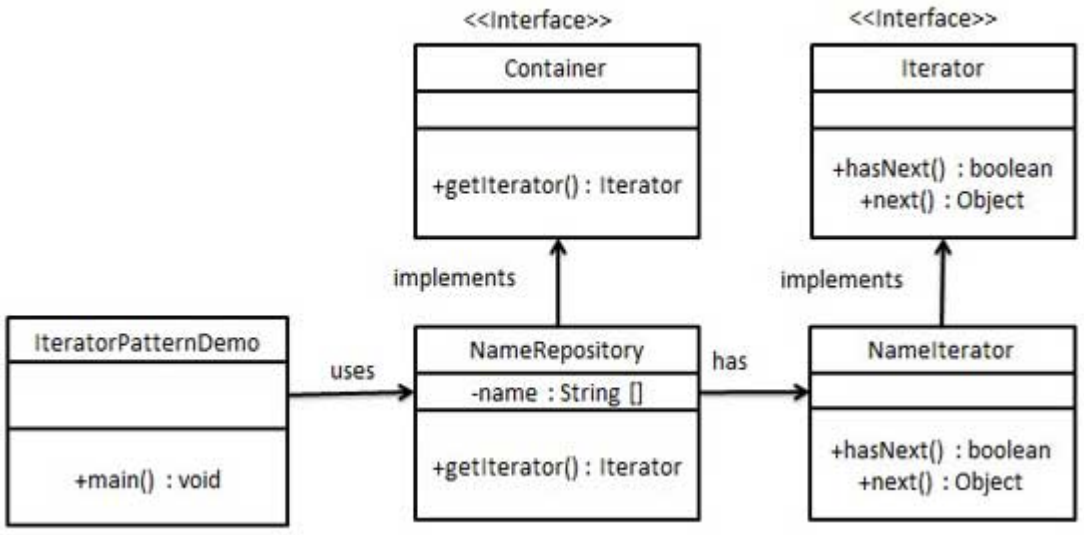
1.5.2 Voorbeeld

Een serveerster wil de menu's van zowel het *Pancake House* (gebruikt een `ArrayList`) als het *Diner* (gebruikt een `Array`) afdrukken. Zonder Iterator moet zij voor elk menu een aparte lus schrijven (één voor de lijst en één voor de array). Met een Iterator kan zij beide menu's op exact dezelfde manier doorlopen.

1.5.3 Stappen

- **Stap 1:** Definieer een `Iterator` interface met methoden zoals `hasNext()` en `next()`.
- **Stap 2:** Maak concrete Iterator-klassen (bijv. `DinerMenuIterator`) die deze interface implementeren voor een specifieke collectie.
- **Stap 3:** Definieer een gemeenschappelijke `Menu` interface die een methode `createIterator()` bevat.
- **Stap 4:** Implementeer de `createIterator()` methode in de concrete klassen (bijv. `DinerMenu`), zodat deze de juiste Iterator teruggeven.
- **Stap 5:** Pas de client (bijv. de `Waitress`) aan zodat deze enkel de `Iterator` interface gebruikt om door de items te lopen, ongeacht de bron.

1.5.4 UML



1.5.5 Code

Maak de oefening(en) op chamilo

1.6 Composite Pattern

Het Composite pattern wordt gebruikt wanneer objecten in een boomstructuur (geheel-deel hiërarchie) georganiseerd zijn.

1.6.1 Gebruik

- Stelt clients in staat om individuele objecten en composities van objecten op een uniforme manier te behandelen.

- Wordt gebruikt om structuren van objecten weer te geven (zoals een menu dat sub-menu's bevat).
- Maakt recursieve structuren mogelijk waarbij een "geheel" zelf weer uit "delen" bestaat.

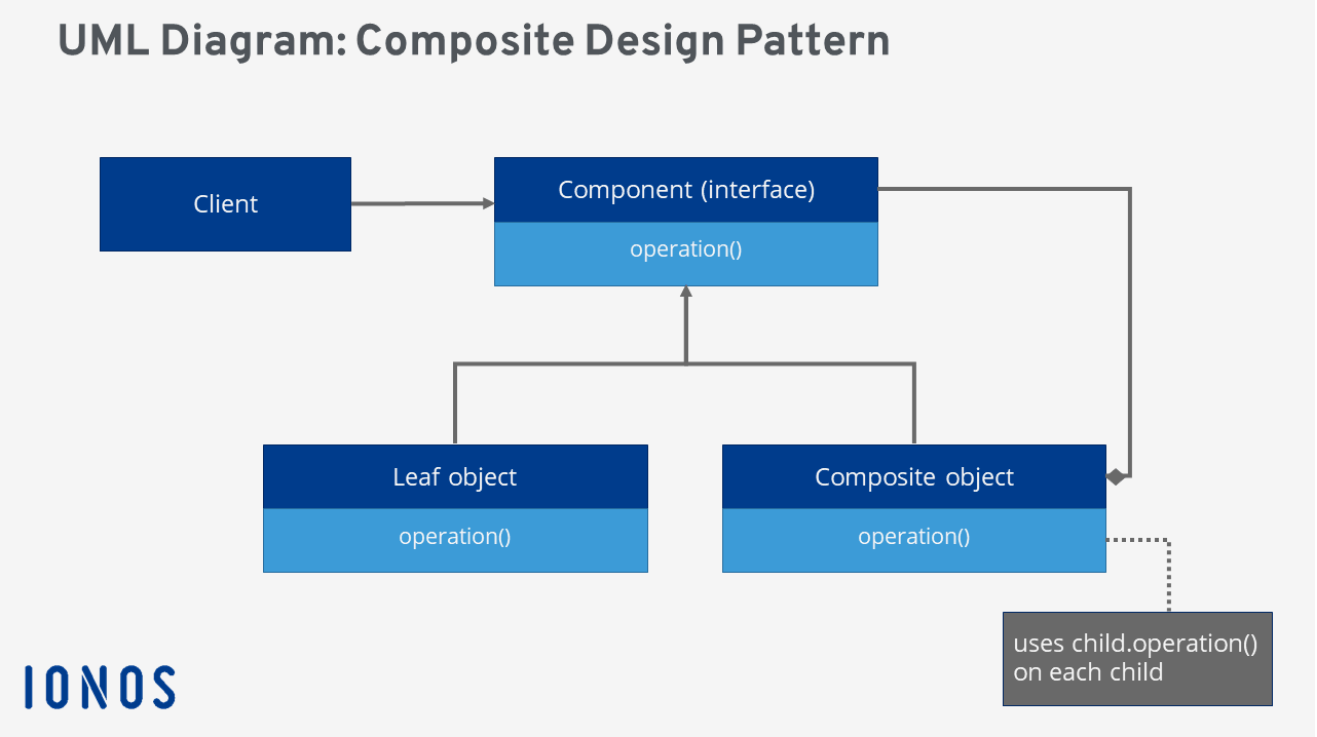
1.6.2 Voorbeeld

Een restaurantmenu kan bestaan uit losse gerechten (`MenuItems`), maar ook uit sub-menu's (bijv. een "Dessert menu" binnen het hoofdmenu). Met het Composite pattern kan de serveerster simpelweg `print()` aanroepen op het hoofdmenu, waarna de hele boomstructuur (inclusief alle sub-menu's en hun gerechten) recursief wordt afgedrukt.

1.6.3 Stappen

- **Stap 1:** Maak een abstracte `Component` klasse (bijv. `MenuComponent`) die de standaardoperaties definieert voor zowel de bladeren als de composities.
- **Stap 2:** Maak de `Leaf` klasse (bijv. `MenuItem`) die de `Component` uitbreidt en de specifieke logica voor individuele elementen implementeert.
- **Stap 3:** Maak de `Composite` klasse (bijv. `Menu`) die ook de `Component` uitbreidt, maar daarnaast een lijst van `Component` kinderen bijhoudt.
- **Stap 4:** Implementeer in de `Composite` klasse methoden om kinderen toe te voegen of te verwijderen (`add()` , `remove()`).
- **Stap 5:** Implementeer de hoofdoperatie (bijv. `print()`) in de `Composite` klasse zodanig dat deze de operatie delegeert aan al zijn kinderen (recursie).

1.6.4 UML



1.6.5 Code

Maak de oefening(en) op chamilo

1.7 Command Pattern

The Command Pattern Explained and Implemented in Java | Behavioral Design Patterns | Geekific
Command Design Pattern: Easy Guide for Beginners

1.7.1 Gebruik

- **Ontkoppeling:** Het scheidt het object dat de aanvraag doet (de *Invoker*) van het object dat weet hoe de aanvraag moet worden uitgevoerd (de *Receiver*).
- **Undo/Redo:** Omdat elk commando een object is, kun je ze opslaan in een geschiedenis (stack) om acties eenvoudig ongedaan te maken.
- **Wachtrijen:** Maakt het mogelijk om acties in te plannen, te loggen of op een later tijdstip uit te voeren.
- **Uitbreidbaarheid:** Je kunt nieuwe commando's toevoegen zonder de bestaande code van de zender of ontvanger aan te passen.

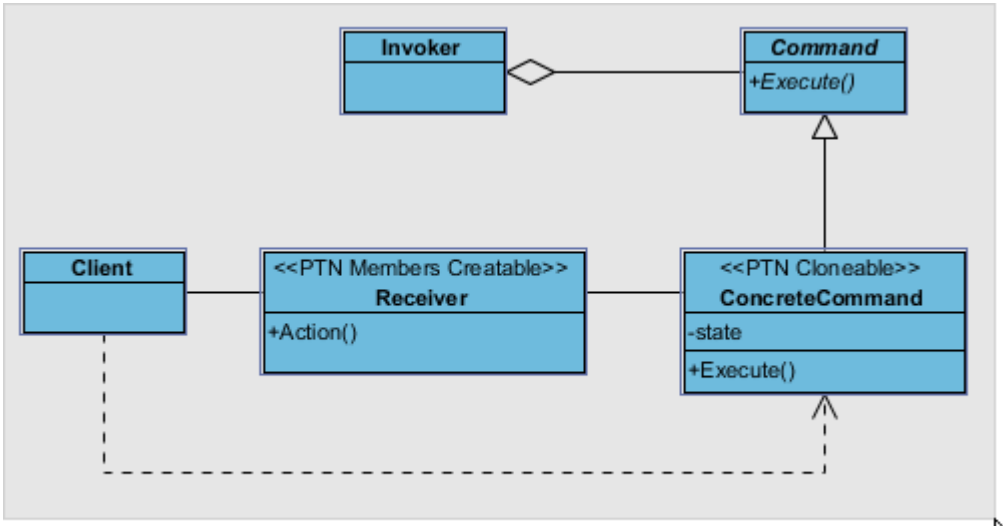
1.7.2 Voorbeeld

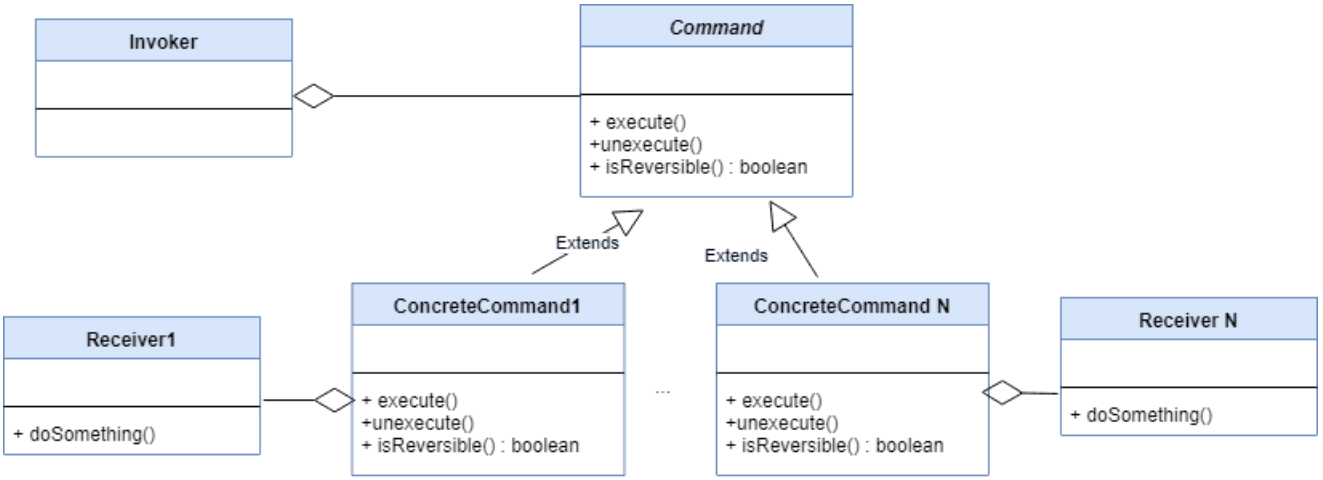
Stel je een **slimme afstandsbediening** voor (de *Invoker*). Deze afstandsbediening heeft knoppen, maar weet niet wat de apparaten in huis precies doen. Wanneer je op een knop drukt, roept de afstandsbediening `execute()` aan op een `Command` object (bijv. `LightOnCommand`). Dit commando weet precies welke lamp (de *Receiver*) hij moet aanschakelen. De afstandsbediening hoeft dus alleen te weten hoe hij een commando moet uitvoeren, niet hoe een lamp werkt.

1.7.3 Stappen

- **Stap 1:** Maak een `Command` interface aan die ten minste één methode definieert: `execute()`. Optioneel kun je ook een `undo()` methode toevoegen.
- **Stap 2:** Maak de `Receiver` klasse(n) (bijv. `Light` of `Stereo`) die de daadwerkelijke business logica bevatten om een taak uit te voeren.
- **Stap 3:** Maak `ConcreteCommand` klassen (bijv. `LightOnCommand`) die de `Command` interface implementeren. Deze klassen houden een referentie bij naar de `Receiver` en roepen de juiste methode aan in hun `execute()` implementatie.
- **Stap 4:** Maak de `Invoker` klasse (bijv. `RemoteControl`). Deze houdt een commando vast en roept op een specifiek moment (zoals bij een knopdruk) de `execute()` methode aan.
- **Stap 5:** In de `Client` code breng je alles samen: je creëert de `Receiver`, koppelt deze aan een `ConcreteCommand`, en geeft dat commando aan de `Invoker`.

1.7.4 UML





1.7.5 Code

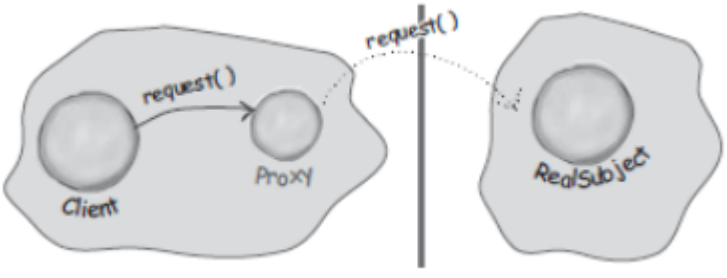
Maak de oefening(en) op chamilo

1.8 Proxy Pattern

[Structural Design Patterns | Geekific]
[Proxy Design Pattern: Easy Guide for Beginners]

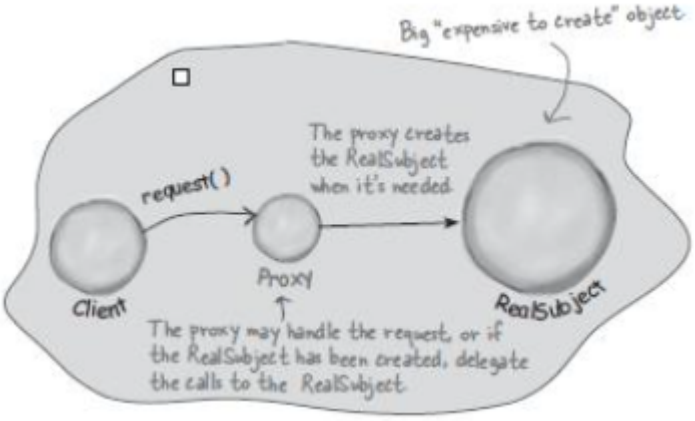
1.8.1 Gebruik

- Remote proxy
  - lokale vertegenwoordiger van een object in een andere JVM



- Virtuele proxy

- vertegenwoordiger van object dat te duur is om zelf te maken.
- fungeert als surrogaat voor object voor en tijdens de creatie
    - daarna delegeert de proxy meteen naar een RealSubject



1.8.2 Voorbeeld

Een `HeavyBook` kan een object zijn dat een compleet e-book van 500MB en duizenden pagina's uit een database moet laden. Als je een lijst van 100 boeken wilt tonen, zou het laden van 100 `HeavyBook` objecten het systeem laten vastlopen.

In plaats daarvan gebruik je `LightBook` (de Proxy). Deze bevat alleen de metadata zoals de naam en auteur. Pas wanneer de gebruiker op "Open boek" klikt en een methode aanroept die de volledige inhoud nodig heeft, zal de `LightBook` intern het `HeavyBook` object aanmaken en de aanroep doorsturen.

1.8.3 Stappen

- **Stap 1:** Maak een interface (bijv. `Book`) die de methoden definieert die zowel de proxy als het echte object moeten hebben (bijv. `getName()`, `getAuthor()`).
- **Stap 2:** Maak de **RealSubject** klasse (bijv. `HeavyBook`) die de interface implementeert. Deze klasse bevat de zware logica of data-loading in de constructor.
- **Stap 3:** Maak de **Proxy** klasse (bijv. `LightBook`) die ook de interface implementeert.
- **Stap 4:** Voeg in de Proxy een referentie toe naar het echte object (bijv. `private HeavyBook realBook;`), maar initialiseer deze nog niet (**null**).
- **Stap 5:** Implementeer de methoden in de Proxy.

1.8.4 UML

Remote proxy:

- geen idee of we dit zelfs moeten kennen tbh
  Virtual proxy:
-

1.8.5 Code

Maak de oefening(en) op chamilo

1.9 Template Method

The Template Method Pattern Explained Implemented in Java | Behavioral Design Patterns | Geekific
Template Method Design Pattern: Easy Guide for Beginners

1.9.1 Gebruik

- **Algoritmen afschermen:** Het definieert de vaste volgorde van een proces in een centrale methode.
- **Code-duplicatie voorkomen:** Gemeenschappelijke stappen van verschillende klassen worden gegeneraliseerd in een abstracte bovenklasse.
- **Hollywoodprincipe:** Volgt het principe "Bel ons niet, wij bellen jou". De high-level component bepaalt wanneer en hoe low-level componenten deelnemen aan de berekening.
- **Flexibiliteit via Hooks:** Maakt gebruik van methoden die subklassen *kunnen* overschrijven (maar niet hoeven) om optioneel gedrag toe te voegen.

1.9.2 Voorbeeld

Bij het bereiden van warme dranken zoals koffie en thee zijn de stappen grotendeels gelijk: water koken, de drank trekken, in een kopje schenken en smaakstoffen toevoegen.

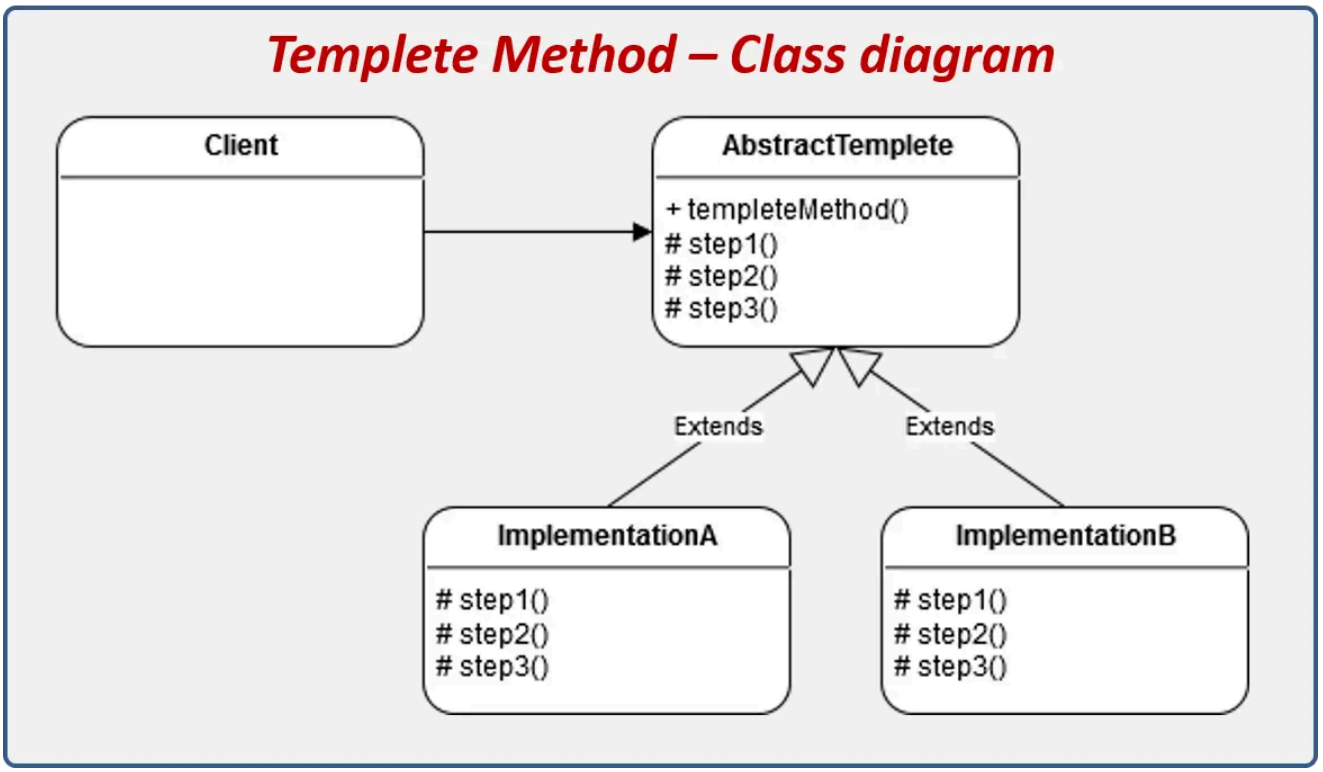
Met het Template Method pattern maak je een abstracte klasse `CaffeineBeverage` met een `final` methode `prepareRecipe()`. Deze methode roept de stappen in de juiste volgorde aan. Stappen zoals "water koken" zijn voor elke drank hetzelfde en worden centraal geïmplementeerd. De specifieke stappen, zoals het type ingrediënt dat moet trekken (`brew`) of de smaakstoffen (`addCondiments`), worden als abstracte methoden overgelaten aan de subklassen `Coffee` en `Tea`.

1.9.3 Stappen

- **Stap 1:** Maak een abstracte klasse (`AbstractClass` , bijv. `CaffeineBeverage`) die het algoritme zal bevatten.
- **Stap 2:** Implementeer de **template method** (bijv. `prepareRecipe()`). Markeer deze als `final` om te voorkomen dat subklassen de volgorde van het algoritme kunnen veranderen.
- **Stap 3:** Definieer de variabele stappen van het algoritme als `protected abstract` methoden (bijv. `brew()`). Deze moeten door de subklassen worden geïmplementeerd.
- **Stap 4:** (Optioneel) Voeg **hooks** toe. Dit zijn concrete methoden met een lege of standaard implementatie die subklassen naar eigen keuze kunnen overschrijven (bijv. `customerWantsCondiments()`).

- **Stap 5:** Implementeer de concrete subklassen (`ConcreteClass` , bijv. `Coffee` of `Tea`) die de abstracte methoden invullen met specifieke business logica.

1.9.4 UML



1.9.5

Maak de oefening(en) op chamilo

2. Java

2.1 Multithreading

2.1.1 Inleiding

- **Thread**: een deel van een programma dat gelijktijdig met andere delen in executie gaat. Het is een sequentiële besturingsstroom — een "lichtgewicht" proces.
- Java heeft ingebouwde primitieven voor multithreading (andere talen zijn afhankelijk van het OS).
- Thread scheduling in Java is wél platformafhankelijk.
- Voorbeeld van multithreading in Java: de **garbage collector**.

2.1.2 Thread Toestanden: Levenscyclus

Toestand	Beschrijving
new	Thread is aangemaakt maar nog niet gestart.
runnable	`start()` is aangeroepen; thread wordt beschouwd als in executie.

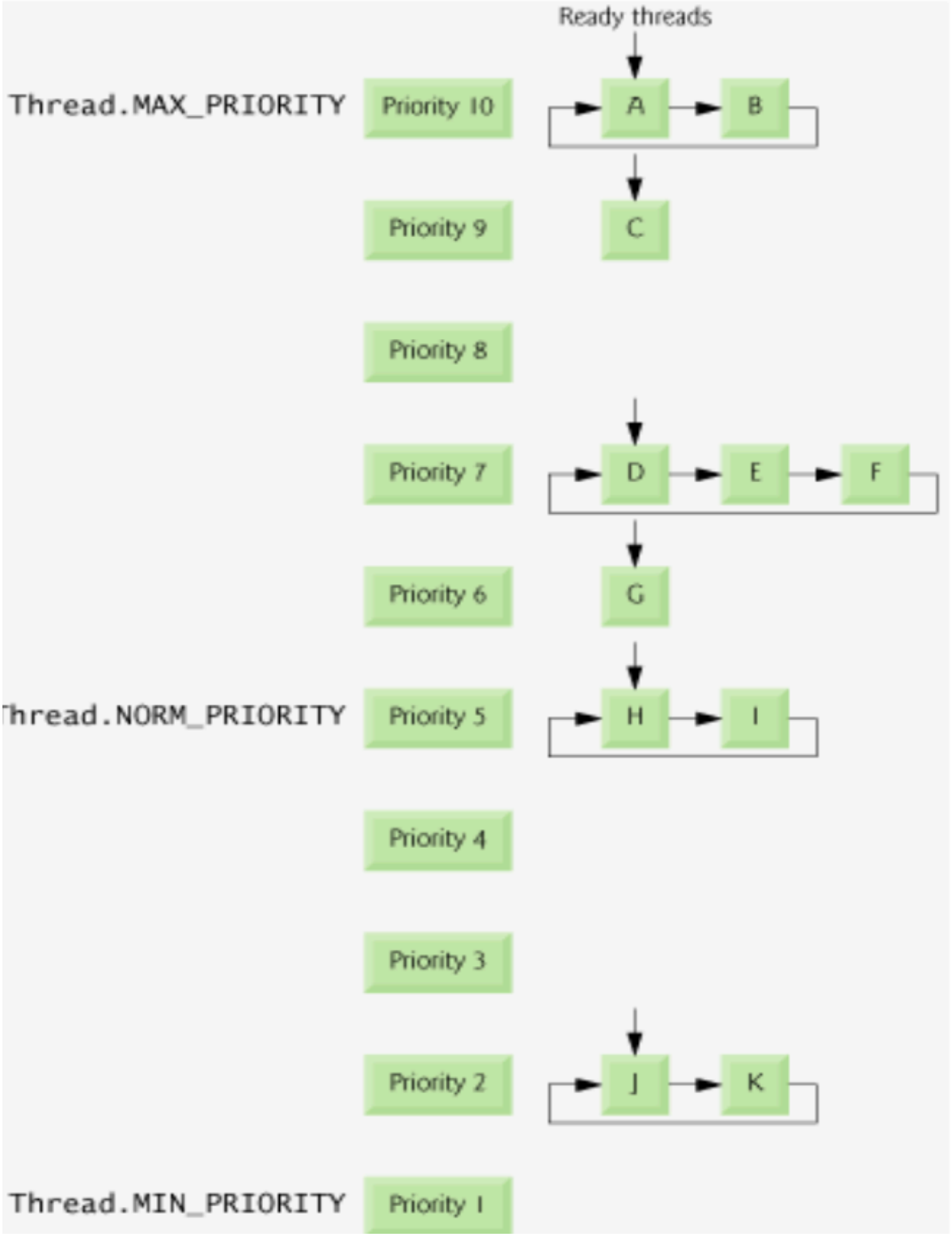
Toestand	Beschrijving
waiting	Wacht op een andere thread (via `wait()` / `notify()`).
timed waiting	Wacht voor een bepaalde tijd (via `sleep()`).
blocked	Geblokkeerd door een I/O-verzoek of een lock.
terminated	Thread is beëindigd (taak volbracht of `exit()`).

Runnable (JVM) = Ready + Running (OS)

- **Ready**: initiële toestand, of na het verlopen van een timeslice.
- **Running**: thread is toegewezen aan een processor (**dispatching**).

> Thread scheduling is een functie van het OS én thread priorities.

2.1.3 Thread Priorities en Scheduling



- **Priority**: indicatie van belangrijkheid (geeft voorrang bij processortijd).
  - `Thread.MIN_PRIORITY` (1) — `Thread.NORM_PRIORITY` (5) — `Thread.MAX_PRIORITY` (10)
- De scheduler zorgt dat de thread met de **hoogste prioriteit** draait.
- Bij gelijke prioriteit: **timeslicing** (elke thread krijgt een quantum).
- Methode `yield()`: laat threads van gelijke prioriteit concurreren.

2.1.4 Creatie en Executie van Threads

```
Starting threads
Threads started, main ends

thread1 going to sleep for 1217
thread2 going to sleep for 3989
thread3 going to sleep for 662
thread3 done sleeping
thread1 done sleeping
thread2 done sleeping
```

```
Starting threads
thread1 going to sleep for 314
thread2 going to sleep for 1990
Threads started, main ends

thread3 going to sleep for 3016
thread1 done sleeping
thread2 done sleeping
thread3 done sleeping
```

2.1.4.1 Via `Runnable` + klasse `Thread`

```
public class PrintTask implements Runnable {
    public void run() {
        // uit te voeren code
        Thread.sleep(sleepTime);
    }
}

Thread t1 = new Thread(new PrintTask("thread1"));
t1.start(); // plaatst thread in Ready toestand
```

2.1.4.2 Via Executor Framework

```
ExecutorService executor = Executors.newFixedThreadPool(3);

executor.execute(new PrintTask("thread1"));

executor.execute(new PrintTask("thread2"));

executor.execute(new PrintTask("thread3"));

executor.shutdown(); // sluit pool af als alle threads terminated zijn`
```

> De `run()` methode wordt automatisch aangeroepen wanneer de thread voor het eerst in running toestand komt. Het programma eindigt pas als **alle threads** terminated zijn.

2.1.5 Thread Synchronisatie

Wanneer meerdere threads een **shared object** kunnen wijzigen, kunnen er problemen optreden.

Oplossing: mutual exclusion / synchronisatie

- Java gebruikt **locks** voor synchronisatie.

- Interface: `Lock` (`java.util.concurrent.locks`)
- Implementatie: `ReentrantLock`

Monitor-mechanisme

- Elk object heeft een **monitor** die slechts één thread tegelijk executierecht geeft (`obtaining the lock`).
- Andere threads komen in **waiting** toestand (lock).
- Bij vrijgave (`release`) krijgt de thread met hoogste prioriteit voorrang (unlock → runnable).

Deadlock preventie

Als een thread `wait()` activeert, zorg dan dat een **andere thread** `notify()` aanroept om terug naar runnable te gaan.

2.1.6 Producer/Consumer Relatie

2.1.6.1 Zonder synchronisatie

Logische fouten zijn mogelijk:

- Consumer leest een waarde die nog niet geschreven is (initial waarde `-1`).
- Producer schrijft meerdere keren voordat Consumer leest.

2.1.6.2 Met synchronisatie (`Lock` + `Condition`)

```
private Lock accessLock = new ReentrantLock();
private Condition canWrite = accessLock.newCondition();
private Condition canRead  = accessLock.newCondition();
private boolean occupied   = false;
```

Producer (`set`):

1. `accessLock.lock()`
2. Zolang buffer vol: `canWrite.await()`
3. Schrijf waarde, zet `occupied = true`
4. `canRead.signal()`
5. `accessLock.unlock()` (in `finally`)

Consumer (`get`):

1. `accessLock.lock()`
2. Zolang buffer leeg: `canRead.await()`
3. Lees waarde, zet `occupied = false`
4. `canWrite.signal()`

5. `accessLock.unlock()` (in `finally`)

> `await()` → thread in **waiting state**, lock wordt vrijgegeven.
> `signal()` → thread terug in **runnable state**.

2.1.7 Producer/Consumer: `ArrayBlockingQueue`

Hiërarchie: `Collection` → `Queue` → `BlockingQueue` → **`ArrayBlockingQueue`**

- `put(value)` : plaatst element achteraan; **blokkeert** als de queue vol is.
- `take()` : haalt eerste element op; **blokkeert** als de queue leeg is.
- Synchronisatie gebeurt **automatisch** — geen `Lock` of `Condition` nodig.
- De array wordt gebruikt als een **statische circulaire buffer**.

```
private ArrayBlockingQueue<Integer> buffer = new ArrayBlockingQueue<>(3);

// Producer
buffer.put(value);

// Consumer
int val = buffer.take();
```

2.1.8 Multithreading met GUI (JavaFX)

- GUI-componenten zijn **niet thread-safe**.
- Slechts één thread mag GUI-componenten aanpassen: de **JavaFX Application Thread**.
- Alle GUI-taken worden in een **queue** geplaatst en sequentieel uitgevoerd (**thread confinement**).

Twee regels bij multithreaded JavaFX:

1. Tijdrovende taken mogen **niet** door de JavaFX Application Thread worden uitgevoerd.
2. JavaFX-componenten mogen **enkel** door de JavaFX Application Thread worden benaderd.

Kleine GUI-updates: `Platform.runLater()`

```
Platform.runLater(new Runnable() {
    public void run() {
        btn.setText("Buy now");
    }
});
```

Grote achtergrondtaken: `javafx.concurrent` pakket

- Interface: `Worker`
- Klassen: `Task` en `Service`

2.1.9 Interface `Callable`

`Runnable.run()` is `void` en kan geen `checked exception` gooien → gebruik `Callable`.

- `Callable<T>` heeft één methode: `call()` — **geeft een waarde terug** en **kan een exception gooien**.
- Uitvoeren via `ExecutorService.submit()` → geeft een `Future<T>` terug.
- `future.get()` blokkeert tot het resultaat beschikbaar is.

```
public class PrimeSearch implements Callable<BigInteger> {
    public BigInteger call() {
        return BigInteger.probablePrime(bitsize, random);
    }
}

Future<BigInteger> result = pool.submit(new PrimeSearch(16));
System.out.println(result.get()); // blokkeert tot klaar
```

2.1.10 Methode `join()`

- `thread.join()` : huidige thread **wacht** tot de opgegeven thread terminated is.
- `thread.join(long millis)` : wacht maximaal het opgegeven aantal milliseconden.
- Beide methoden gooien een `InterruptedException`.

```
producer.join(); // Consumer wacht tot Producer klaar is
```

2.1.11. Daemon Threads

- Een **daemon thread** draait als ondersteuning van andere threads.
- Verhindert **niet** dat het programma eindigt (in tegenstelling tot gewone threads).
- Voorbeeld: Java's garbage collector.
- Markeren als daemon: `thread.setDaemon(true)` — **vóór** `start()` aanroepen.
- Controleren: `thread.isDaemon()`

2.1.12 Stop, Suspend en Resume (deprecated)

`stop()`, `suspend()` en `resume()` zijn **deprecated**.

Stoppen:

```
while (running) {
    // doe werk
}
// stop door running = false te zetten
```

Pauzeren:

```
while (suspended) {
    wait();
}
```

2.1.13 `volatile`

- Een `volatile` variabele kan door meerdere threads gelezen en bijgewerkt worden **zonder locks**.
- Wijzigingen door één thread zijn **onmiddellijk zichtbaar** voor alle andere threads.
- Vaak gebruikt voor een **vlag** (bijv. om een lus te stoppen).

```
private volatile boolean flag = false;
// Geen Lock nodig voor eenvoudige lees/schrijf operaties
```

2.1.14 Parallel Streams

- Standaard zijn Java Streams **sequentieel**.
- Door `.parallel()` toe te voegen, worden bewerkingen verdeeld over meerdere CPU-cores.

```
// Sequentieel
LongStream stream1 = Arrays.stream(values);
LongSummaryStatistics stats1 = stream1.summaryStatistics();

// Parallel
LongStream stream2 = Arrays.stream(values).parallel();
LongSummaryStatistics stats2 = stream2.summaryStatistics();
```

> In het voorbeeld (100 miljoen elementen): sequentieel ~534ms, parallel ~107ms — een factor ~5 sneller.